

Context-Aware Exploration in Reinforcement Learning Using Large Language Models *Supplementary Material*

This document provides supplementary material accompanying our main paper submitted to [ICTAI, 2026].

Contents

S1 Environment and Experimental Details	2
S2 LLM-DQN Prompts	3

S1 Environment and Experimental Details

This appendix provides additional details about the benchmark environments used in our experiments. We evaluate the proposed approach on a diverse set of reinforcement learning tasks.

CartPole-v1: The CartPole environment is a classic control problem where a pole is attached to a cart moving along a one-dimensional track. The objective is to apply forces to the cart so that the pole remains upright for as long as possible. In the CartPole-v1 setting, an episode terminates after 500 time steps.

MountainCar-v0: The MountainCar environment is a classical reinforcement learning problem where an underpowered car must drive up a steep mountain. Because the engine power is insufficient to directly climb the hill, the agent must first move away from the goal to build momentum before attempting to reach the top. The goal is to reach the flag located at the top of the mountain. This environment requires strategic exploration because the optimal solution initially moves away from the target. In our experiments, we adopt a modified reward formulation where successful completion of the task corresponds to a return threshold of +150, which is used as the convergence criterion during evaluation.

LunarLander: The LunarLander environment models a two-dimensional landing problem where a spacecraft must safely land on a designated landing pad. The reward function encourages stable landings near the target location while penalizing crashes and excessive fuel usage. LunarLander is a more complex control task that requires coordinated thrust control and stable descent dynamics.

CliffWalking (Deterministic and Stochastic): The CliffWalking environment is a grid-world navigation task where the agent must move from a start state to a goal state while avoiding falling off a cliff. The environment is typically represented as a two-dimensional grid in which stepping into the cliff results in a large negative reward and resets the agent to the start position. The deterministic version transitions exactly according to the chosen action, while the stochastic variant introduces randomness in action execution, making the navigation task more challenging. This environment is commonly used to evaluate exploration strategies and risk-aware policies.

FrozenLake (Deterministic and Stochastic): The FrozenLake environment is another grid-world navigation problem where the agent must traverse a frozen surface to reach a goal location while avoiding holes in the ice. In the deterministic variant, actions result in predictable transitions, whereas the stochastic version introduces slippage, causing the agent to move in unintended directions with some probability. This environment evaluates the agent’s ability to handle uncertainty and delayed rewards in discrete decision-making tasks.

S2 LLM-DQN Prompts

LLM-DQN Prompt for Cart Pole

%D: Environment description

You are a smart environment explorer working on the Cart Pole environment. The goal of the agent is to balance a pole on a cart. However, your goal is to generate a rule-based exploration policy to help the DQN agent learn faster. The observation space is 4-dimensional: x (cart position), v (cart velocity), θ (pole angle), ω (pole angular velocity). The action space is discrete: 0 (push cart left), 1 (push cart right).

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return (most recent policy, evaluated over 50 episodes): {eval_r_for_llm:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage Statistics (per rule-defined region):

{coverage_info}

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

History of Previous Policies, Evaluation Returns, and Coverage Statistics:

{history_text}

Task:

Use both the evaluation return and the region coverage statistics to generate the next exploratory policy. The objective is not exploration itself, but to improve the DQN agent's learning performance and achieve higher evaluation returns. Rules must be non-conflicting: use mutually exclusive conditions, or rely on ordering, where the first matching rule applies.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "policy_type": "rules",
  "name": "policy_ep_{episode}",
  "rules": [
    {"if": [ {"feature": "theta", "op": "<", "value": -0.1}, ... ], "then": 1},
    ...
  ],
  "default_action": 0,
  "notes": "string"
}
```

LLM-DQN Prompt for Lunar Lander

%D: Environment description

You are a smart environment explorer working on the Lunar Lander environment, where the agent must safely land the spacecraft between the flags while staying stable during descent. Your role is to generate a rule-based exploration policy that helps the DQN agent learn faster. The observation space is 8-dimensional: x, y (position), v_x, v_y (velocity), θ (angle), ω (angular velocity), and left/right leg contact. The four actions are: 0 (do nothing), 1 (fire left engine), 2 (fire main engine), 3 (fire right engine). The environment rewards proximity to the pad, low speed, low tilt, and leg contact, while penalizing engine use, tilt, and speed, with a bonus of +100 for a safe landing or a penalty of -100 for a crash. A successful landing is typically marked by x, v_x, v_y, θ , and ω all near 0, with at least one leg touching down.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return (of the most recent suggested policy, over 10 greedy evaluation episodes): {eval_r_for_llm:2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage statistics indicate the percentage of visited states belonging to each rule-defined region during the 10 evaluation episodes:

{coverage_info}

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

For each previous policy, its rules, evaluated return, and coverage statistics are provided:

{history_text}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

Each trajectory is summarized by sampling states from beginning and middle of the episode, while preserving the final states to capture terminal behavior (successful landing or crash).

{trajectory_text}

Task: Use the evaluation returns, coverage statistics, and trajectories together, treating trajectories as the primary evidence for deciding which rules to modify. Preserve rules from successful or high-return policies, modify only the smallest number of rules (preferably one or two) unless trajectories reveal a new repeated failure pattern, and reuse successful rules from previous policies where appropriate.

%Formatting Instructions

Output JSON only:

```
{
  "policy_type": "rules",
  "name": "policy_ep_{episode}",
  "rules": [
    {"if": [ {"feature": "vy", "op": "<", "value": -0.5}, ... ], "then": 1},
    ...
  ],
  "default_action": 0,
  "notes": "string"
}
```

LLM-DQN Prompt for MountainCar

%D: Environment description

You are controlling a car in Mountain Car environment. The goal is to reach the flag at position > 0.5 . Actions: 0 (left), 1 (nothing), 2 (right). Gravity pulls the car down, and the engine is too weak to climb directly, so the agent must build momentum by rocking back and forth. The task is to define a non-conflicting policy that splits the state space (velocity, position) into a Low Energy state, which needs momentum building, and a High Energy state, which should drive up the hill.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Eval Score (Greedy): {eval_score:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Low Energy Region: {coverage_info["low_energy"]} %

High Energy Region: {coverage_info["high_energy"]} %

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

History of Policies (each with its splits, average return, epsilon, and coverage statistics):
{history_json}

% $\mathcal{T}_m$: Representative evaluation trajectories

Train Trajectories (Exploration), summarized as return and length per run:
{traj_summary}

Task:

Use both the evaluation score and the coverage statistics to update the state-space split. Provide new values for the velocity split, position split, and the actions assigned to the low- and high-energy regions.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "vel_split": float,
  "pos_split": float,
  "action_low_energy": int,
  "action_high_energy": int,
  "reasoning": "string"
}
```

LLM-DQN Prompt for FrozenLake(Stochastic) Environment

%D: Environment description

You are an expert explorer helping an agent with exploration on FrozenLake-v1 (4×4 grid), where the agent can take action 0, 1, 2, or 3, over 16 discrete states arranged as:

```
S F F F
F H F H
F F F H
H F F G
```

S is the start, F is frozen surface, H is a hole (episode ends, 0 reward). The environment is slippery: each action results in one of three neighboring directions with equal probability, rather than the intended direction. Action mapping: 0 = LEFT, 1 = DOWN, 2 = RIGHT, 3 = UP. The task is to propose a policy that takes safe actions to maximize the average return.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Current Average Evaluation Return: {avg_eval_reward:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage Statistics:

LEFT Region: {coverage_info["left_pct"]:.2f}%

DOWN Region: {coverage_info["down_pct"]:.2f}%

RIGHT Region: {coverage_info["right_pct"]:.2f}%

UP Region: {coverage_info["up_pct"]:.2f}%

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

{past_policies_json}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

{history_json}

Task: The trajectories are the primary source of evidence. Preserve actions that consistently appear in successful trajectories; modify only actions with repeated evidence of failure. Preserve the policy with minimal changes if it improves on the previous return, and correct only the failing states if it degrades.

%Formatting Instructions

Output JSON only:

```
{
  "policy": [16 integers in [0,3]],
  "reasoning": "string"
}
```

LLM-DQN Prompt for CliffWalking(Deterministic) Environment

%D: Environment description

You are an expert explorer helping an agent with exploration on CliffWalking-v1 (is_slippery=False). The environment is deterministic: every action always produces the intended movement. The grid is 4×12 , giving states 0–47. Rewards are -1 per step and -100 for entering the cliff (states 37–46), after which the agent resets to the start state 36. The agent moves using four actions (0: UP, 1: RIGHT, 2: DOWN, 3: LEFT). Do not enter cliff states; focus especially on cliff states (37–46), states near the cliff (25–35), and the start state (36). Improve the exploration policy using the provided trajectories, coverage statistics, and previous policy performance.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return: {avg_eval_reward:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage statistics indicate the percentage of states visited during the {n_trajectories_llm} greedy evaluation trajectories that fall under each action region defined by the current exploration policy.

Action {region}: {pct:.2f}% (repeated for each action present in the current partial policy)

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

History of Previous Policies and Evaluation Returns:

{past_policies.json}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

Representative Greedy Evaluation Trajectories:

{history.json}

Task:

Identify where the agent is repeatedly failing or getting stuck, focusing especially on states near the cliff. Improve only the critical states and avoid repeating unsuccessful policy modifications; if similar states were suggested previously without improving the evaluation return, modify different states instead. The objective is to reach the goal using the shortest safe path while avoiding the cliff and avoiding loops or oscillations between states. Prefer moving RIGHT toward the goal whenever safe, and use UP only when necessary to avoid the cliff. Output only a partial policy: do not assign actions to all states, and do not assign the same action to all states; modify only 2–5 of the most problematic states, and keep the partial policy as small as possible.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "partial_policy": {state: action},
  "reasoning": "string"
}
```

LLM-DQN Prompt for CliffWalking(Stochastic) Environment

%D: Environment description

You are an expert explorer helping an agent with exploration on CliffWalking-v1 (is_slippery=True). The grid is 4×12 , giving states 0–47. Rewards are -1 per step and -100 for entering the cliff (states 37–46), after which the agent resets to the start state 36. Since the environment is stochastic, each action has a probability of sliding into one of three possible directions rather than always producing the intended movement (e.g., action 0 may result in left, up, or right; action 1 in up, right, or down; action 2 in right, down, or left; action 3 in down, left, or up). Never enter cliff states; focus especially on cliff states (37–46), states near the cliff (25–35), and the start state (36). Identify failure patterns from trajectories, avoid the cliff aggressively, and prefer safe paths. Improve only critical states.

% $\hat{J}(\pi_{\text{DQN}})$: Empirical evaluation of the current DQN policy

Average Evaluation Return: {avg_eval_reward:.2f}

Exploration Rate (ϵ): {current_epsilon:.2f}

%C: Coverage statistics of the current LLM exploration policy

Coverage statistics indicate the percentage of states visited during the 10 greedy evaluation trajectories that fall under each action region defined by the current exploration policy.

Action {region}: {pct:.2f}% (repeated for each action present in the current partial policy)

% $\mathcal{H}_p$: Previous LLM policies and their evaluation returns

Past Policies and their performance:

{past_policies.json}

% $\mathcal{T}_m$: Representative greedy evaluation trajectories

Current Trajectories of DQN Learning:

{history.json}

Task:

Identify where the agent is failing, focusing especially on states near the cliff (35–46). Improve only critical states, and avoid repeating bad policies; if similar states were already suggested in past policies and did not improve returns, choose different states instead. The goal is located at the far right of the grid; the agent should prioritize reaching the goal in the minimum number of steps while avoiding loops or oscillations between states. Prefer moving RIGHT unless it leads directly into a cliff. Use UP only to escape danger, not as a default action. Do not assign the same action (e.g., LEFT or UP) to all states.

%Formatting Instructions

Output JSON only in the following format:

```
{
  "partial_policy": {state: action},
  "reasoning": "string"
}
```